

Exploration d'environnements souterrains par déploiement et réorganisation d'une chaîne de drones

Mathis Fleuriel^a
mathis.fleuriel@insa-lyon.fr

Elena Vanneaux^b
elena.vanneaux@ensta.fr

David Filliat^b
david.filliat@ensta.fr

Olivier Simonin^a
olivier.simonin@insa-lyon.fr

^aINSA Lyon, Inria Lyon, CITI Lab., Villeurbanne, France

^bU2IS, ENSTA, Institut Polytechnique de Paris, France

Résumé

Cet article porte sur l'exploration autonome d'environnements souterrains inconnus à l'aide d'un essaim de drones. Dans ce cadre les communications sans fil classiques sont considérées indisponibles car trop limitées. Nous proposons de déployer une chaîne de drones connectés visuellement qui se maintient et se réorganise par des communications locales basées sur la vision. Nous définissons un algorithme d'exploration ainsi que les comportements des drones. Ils reposent notamment sur la distinction de deux rôles, "Leader" et "Follower". Nous évaluons le modèle proposé sur différents scénarios avec un outil de simulation réaliste intégrant un modèle de dynamique des drones.

Mots-clés : Exploration d'environnements, Essaim de drones, Communications visuelles

Abstract

This article focuses on the autonomous exploration of unknown underground environments using a swarm of drones. In this context, conventional wireless communications are considered unavailable because they are too limited. We propose to deploy a chain of visually connected UAVs that maintain and reorganise themselves using local vision-based communications. We define an exploration algorithm and the relative drone behaviours. These are based in particular on the distinction of two roles, 'Leader' and 'Follower'. We evaluate the proposed model on different scenarios with a realistic simulation tool using a UAV dynamics model.

Keywords: Environment exploration, Swarm of drones, Visual communications

1 Introduction

Les robots aériens autonomes (UAV) sont de plus en plus utilisés dans des applications critiques, telles que les opérations de recherche et

de sauvetage (SAR) [9]. Cependant, le déploiement de drones dans des environnements complexes, en particulier sous terre, comme dans les mines [12], les passages souterrains et les grottes [14], reste un défi [3, 6]. Un espace souterrain/intérieur présente un ensemble de propriétés qui compliquent l'autonomie des robots : ils peuvent être vastes, particulièrement étroits et confinés, multi-chemins et avec des boucles [4], monotones rendant difficile la perception, et privés de communications sans fil [13].

Nous proposons de déployer une chaîne de mini-drones, de sorte qu'ils servent de relais entre un drone d'exploration (appelé leader) et un opérateur à l'entrée. En volant de manière autonome, mais en gardant un contact visuel avec son prédécesseur, chaque drone peut estimer sa position relative et communiquer des informations aux autres drones par le biais de signaux visuels. Cette approche ne nécessite aucun système de localisation ou de cartographie (comme un algorithme SLAM visuel) et, de ce fait, (1) elle s'adapte naturellement à la topographie d'un environnement souterrain, (2) elle fonctionne avec des drones disposant d'une mémoire et d'une puissance de calcul limitées, et (3) elle ne nécessite pas l'établissement d'un réseau de communication sans fil (ce qui serait complexe dans des environnements souterrains [13]). Le déploiement de la chaîne de drones repose sur une approche de formation type leader-follower [15], tandis que les agents utilisent des capteurs LiDAR pour estimer les obstacles et les positions de leurs voisins.

Les principales contributions de ce travail sont :

- Extension de l'idée du déploiement d'une chaîne de mini-drones à tout environnement intérieur (introduite eg. dans [11]).
- Proposition d'un algorithme multi-agent garantissant une exploration complète d'un environnement inconnu formé de tunnels.
- Évaluation des principales fonctionnalités à l'aide d'un simulateur réaliste.

L'article débute par un état de l'art en section 2 sur l'exploration d'environnements inconnus par une flotte de drones, puis nous présentons le problème traité et les hypothèses considérées en section 3. La section 4 présente notre approche en détail en considérant d'abord l'exploration par un seul agent avant de détailler la stratégie et les comportements du déploiement d'une chaîne d'agents. Nous présentons le simulateur et les premiers résultats expérimentaux en section 5 avant de conclure en section 6.

2 Etat de l'art

L'exploration des grottes ou des mines à l'aide de systèmes autonomes a récemment suscité un vif intérêt de la part de la communauté scientifique. Accélérés par le lancement du DARPA Subterranean Challenge, les efforts de recherche dans le monde entier portent sur le défi particulier de l'autonomie souterraine à long terme [14, 3, 13]. De nombreuses équipes s'appuient sur des robots terrestres [3, 13]. Cependant, le déploiement de ces robots dans des environnements complexes - souvent vastes, sombres, glissants et particulièrement étroits - pose d'importants problèmes. En revanche, les drones miniatures offrent une alternative plus flexible et plus efficace [14, 6].

Contrairement aux robots terrestres, les mini-drones ne peuvent pas transporter de câble pour rester en contact avec l'opérateur. Ils doivent donc s'appuyer sur une liaison radio, qui fonctionne bien dans les environnements ouverts, mais qui rencontrent des problèmes importants dans les environnements souterrains. Sous terre, la communication radio n'est efficace que si émetteur et récepteur sont en ligne de vue. À chaque tournant de couloir, le signal est fortement affaibli ou perdu. Pour résoudre ce problème, les auteurs de [13, 14] ont proposé d'établir un réseau maillé statique pour connecter les robots à la station de base. Ce réseau se compose du nœud de la station de base, des nœuds des robots et des nœuds de communication WiFi transportés par les drones [13, 14].

Dans [11], les auteurs proposent de déployer une chaîne de drones, où chaque drone sert de relais entre un drone d'exploration (appelé leader) et un opérateur à l'entrée. Cette approche établit un réseau plus flexible et dynamique, permettant la communication par des signaux visuels au lieu de s'appuyer sur des technologies sans fil. Cependant, la méthode proposée est limitée à l'exploration d'un seul tunnel, alors que les structures souterraines sont généralement plus com-

plexes [4] et comprennent des embranchements, des intersections, des impasses et des boucles.

Cet article explore un scénario dans lequel un agent leader guide l'exploration tandis que d'autres forment une chaîne reliant le leader à la base. Deux algorithmes clés d'exploration par un seul agent existent : le suivi de mur "par la main gauche" qui assure une couverture complète uniquement dans les labyrinthes simplement connectés, et l'Algorithme de Trémaux, qui fonctionne dans tous les labyrinthes structurés mais nécessite le marquage des passages déjà explorés [2]. Mentionnons que des travaux pour l'exploration multi-agent ont repris ce principe en permettant la détection de boucle par marquage de l'environnement, comme par exemple dans [8]. Dans notre approche, la connectivité visuelle de la chaîne de drones permet d'éviter de marquer l'environnement : nous nous assurons qu'à chaque intersection entre la base et le leader, il y a un drone suiveur, ce qui indique que des passages ont déjà été empruntés. Cela se rapproche des techniques de détection de boucles par rencontre aux sommets d'un arbre d'exploration, comme proposé dans l'exploration multi-agent centralisée de [10].

Nous créons ainsi un graphe topologique dynamique [10][16], dans lequel les agents sont placés dans les jonctions (ainsi qu'aux limites de la connectivité visuelle [7]). Cela nous permet de proposer un algorithme d'exploration complet pour toute forme de souterrains et d'être capable de remonter au leader sans cartographier l'environnement. Pour déployer la chaîne, nous nous inspirons des approches de formation leader-suiveur [15, 5], tout en accordant une attention particulière au maintien de la connectivité visuelle aux intersections. Cette proposition est développée dans le cadre plus large du projet VORTEX¹.

3 Hypothèses sur le problème et les agents

Dans cette courte section, nous précisons les hypothèses du problème traité dans cet article, à savoir l'exploration complète d'environnements souterrains inconnus par une flotte de drones :

1. Les environnements à explorer sont composés de couloirs étroits toujours traversables, d'intersections et d'impasses (voir illustrations).

1. ANR VORTEX 2023-28, référence ANR-23-IAS2-0005

2. Les drones possèdent un capteur permettant de connaître leur distance aux murs sur 360°.
3. Les communications entre drones sont limitées à des signaux visuels entre voisins.
4. Les drones possèdent un capteur permettant de percevoir les drones directement voisins.
5. Les drones ne font pas de cartographie de l'environnement et ne possèdent pas de système de localisation globale.
6. On considère qu'il n'y a pas de limite sur le nombre de drones disponibles.

Dans un cadre concret, où le nombre de drones est limité (hypothèse 6 non vérifiée) la profondeur d'exploration sera restreinte à la taille maximale de la chaîne d'agents.

La tâche est considérée réalisée lorsque tout l'environnement a été visité. Un critère essentiel est le temps, qui idéalement doit être minimal.

4 Exploration fondée sur une chaîne de drones

Pour réaliser la tâche d'exploration sous les contraintes présentées en section 1, nous avons considéré l'emploi d'une stratégie par déploiement d'une chaîne de drones tous connectés visuellement par au moins un proche voisin (car communications radio/wifi impossibles).

Avant de développer une telle solution (en section 4.2) on se restreint dans un premier temps à l'exploration avec un seul drone pour se focaliser sur la stratégie d'exploration.

4.1 Exploration avec un seul agent

Dans ce cas particulier d'un drone seul, il est évident que le drone ne maintiendra pas de lien permanent avec l'entrée (ce que permettra la chaîne dans la prochaine section).

Algorithm 1 Exploration par un agent seul

```

1: procédure SUIVRE_MUR_GAUCHE
2:   Tant que non agent retourné base faire
3:     Si rencontre impasse alors
4:       fonction DEMI-TOUR
5:     Si rencontre intersection alors
6:       fonction TOURNER(branche_G)
7:     fonction AVANCER_BRANCHE
8:   Exploration terminée
  
```

Note : branche_G = branche la plus à gauche pour un agent à une intersection (voir figure 1).

Nous proposons une solution inspirée de la méthode de résolution d'un labyrinthe sans boucles

dite "méthode de la main droite (ou gauche)", qui consiste à faire longer un mur par sa main sans la décoller. La proposition est donnée par l'algorithme 1. Dans notre cas, l'agent va réaliser un suivi de mur à gauche. Ce qui signifie, dans le contexte de nos environnements, qu'à chaque intersection l'agent prendra la branche de l'intersection la plus à sa gauche, comme explicité figure 1. Et, une fois arrivé dans une impasse, l'agent réalisera un demi tour pour revenir à la dernière intersection détectée dont toutes les branches n'ont pas encore été explorées. L'exploration se termine si toutes les branches de toutes les intersections ont été explorées, donc que l'agent est revenu à la base.

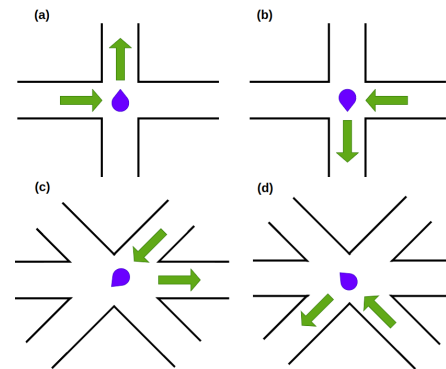


Figure 1 – Illustration du choix de la branche la plus à gauche pour un agent sur deux types d'intersections : (a) et (b) pour une intersection en croix, (c) et (d) pour une intersection en étoile.

En utilisant la stratégie d'exploration présentée dans l'algorithme 1, tous les environnements ne possédant pas de boucles peuvent être complètement explorés [2]. Par contre, pour les environnements contenant des boucles, certaines branches risquent de ne pas être explorées par l'agent, comme présenté sur la figure 2. Une solution bien connue est l'algorithme de Tremaux [2], qui consiste à marquer les passages pris aux intersections, pour détecter une boucle, et dans ce cas faire marche arrière jusqu'à la précédente intersection, pour chercher une nouvelle branche à explorer. Dans cette étude, nos hypothèses de travail sont de ne pas cartographier l'environnement ni marquer l'environnement (ce qui reste délicat à envisager avec des drones). Pour cette raison, nous proposons dans la prochaine section de déployer une chaîne de drones qui va permettre de facilement détecter une boucle lorsque la tête de la chaîne rencontre un de ses éléments.

Il est important de noter que l'Algorithme 1 va être utilisé dans la chaîne par le drone ayant le rôle de leader car il va guider la stratégie d'exploration.

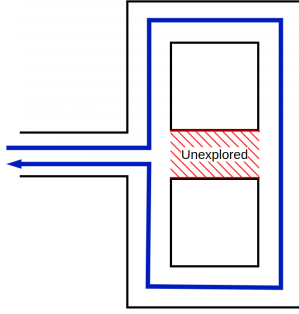


Figure 2 – Environnement en double boucles. La branche faisant la jonction entre les deux boucles reste inexplorée en suivant une stratégie de suivi du mur gauche réalisé par un unique agent.

4.2 Exploration avec une chaîne d'agents

On considère désormais une chaîne d'agents non limitée en nombre. C'est à dire qu'il y a toujours un agent disponible pour venir s'ajouter à la chaîne (à l'entrée de l'environnement, appelée base).

L'idée générale est d'avoir un drone leader qui guide le déploiement d'une chaîne de drones derrière lui. Avant de donner le détail de son comportement, précisons **les hypothèses** sur les comportements des agents formant la chaîne :

1. Deux rôles sont possibles : Leader ou Follower. A tout instant, un seul agent peut être leader, les autres agents sont des followers (ie. suiveur de l'agent le précédent).
2. La chaîne de drones assure la propriété de connectivité (ou connexité), qui garantie la propagation de signaux le long de la chaîne.
3. Durant le déploiement, il y a toujours un agent présent en chaque intersection utilisée par la chaîne.
4. Il y a en permanence un drone stationnaire à l'entrée de l'environnement prêt à entrer.

L'algorithme 2 présente la stratégie d'exploration guidée par le leader. C'est une vision centralisée, qui permet de comprendre le changement de leader lorsque la chaîne rencontre une impasse ou une boucle (c'est à dire elle-même). En section 5 nous présenterons les comportements agents (leader et follower) permettant d'en avoir une version décentralisée.

Dans cet algorithme, la variable "AgentLeader" représente l'agent de la chaîne qui a le rôle de leader. Par défaut, l'agent leader avance dans une branche et se tourne vers la branche la plus à gauche quand une intersection est rencontrée (reprenant la stratégie donnée dans l'Algo-

Algorithm 2 Exploration guidée par un leader

```

1: procedure EXPLORATION_LEADER
2:   AgentLeader  $\leftarrow$  Agent1
3:   Tant que true faire
4:     Si AgentLeader arrive impasse|boucle alors
5:        $f \leftarrow$  IntersectionPrec(AgentLeader)
6:       Tant que non nouveau leader faire
7:         Si f à la base alors
8:           AgentLeader  $\leftarrow$  f
9:           Exploration terminée
10:        sinon
11:          Si BrancheInexplorée(f) alors
12:            AgentLeader  $\leftarrow$  f
13:          sinon
14:             $f \leftarrow$  IntersectionPrec(f)
15:        Si AgentLeader rencontre intersection alors
16:          fonction TOURNER_G(AgentLeader)
17:          fonction AVANCE_BRANCHE(AgentLeader)

```

rithme 1). Lorsque l'agent leader rencontre une impasse ou une boucle, il faut changer d'agent leader pour étirer la chaîne depuis une intersection possédant une branche non explorée. Ce processus (lignes 4-14) consiste à attribuer le rôle "AgentLeader" à un agent follower d'une intersection en amont du leader. Pour cela un mécanisme de "backtracking" permet de remonter aux intersections précédentes. La fonction IntersectionPrec(f) renvoie l'agent situé à l'intersection précédente de celle de l'agent f. Le backtracking s'arrête si la fonction BrancheInexplorée(f) renvoie "Vrai", c'est à dire s'il y a au moins une branche autour de l'agent f qui n'a pas encore été explorée (ligne 11).

Notons que lorsqu'une reconfiguration est réalisée, le nouveau leader va d'abord être suivi par les agents présents dans la branche qui vient d'être explorée. Ainsi le bout de la chaîne devient momentanément un Y, formé de 2 sous-chaînes connectées à l'intersection du nouveau leader. Une telle configuration est illustrée en figure 6.

La propriété de maintien de connectivité de la chaîne nous permet de détecter les boucles. En effet, la perception par le leader d'un drone devant lui, comme une marque dans l'algorithme de Trémaux [2], indique que la chaîne s'est déployée dans une boucle de l'environnement. De plus, le déploiement de cette chaîne systématiquement à gauche permet aux agents, qui ne font pourtant pas de cartographie, de savoir si toutes les branches d'une intersection ont été explorées.

5 Comportement des agents

Maintenant que la stratégie d'exploration par le déploiement d'une chaîne de drones a été pré-

Contribution originale

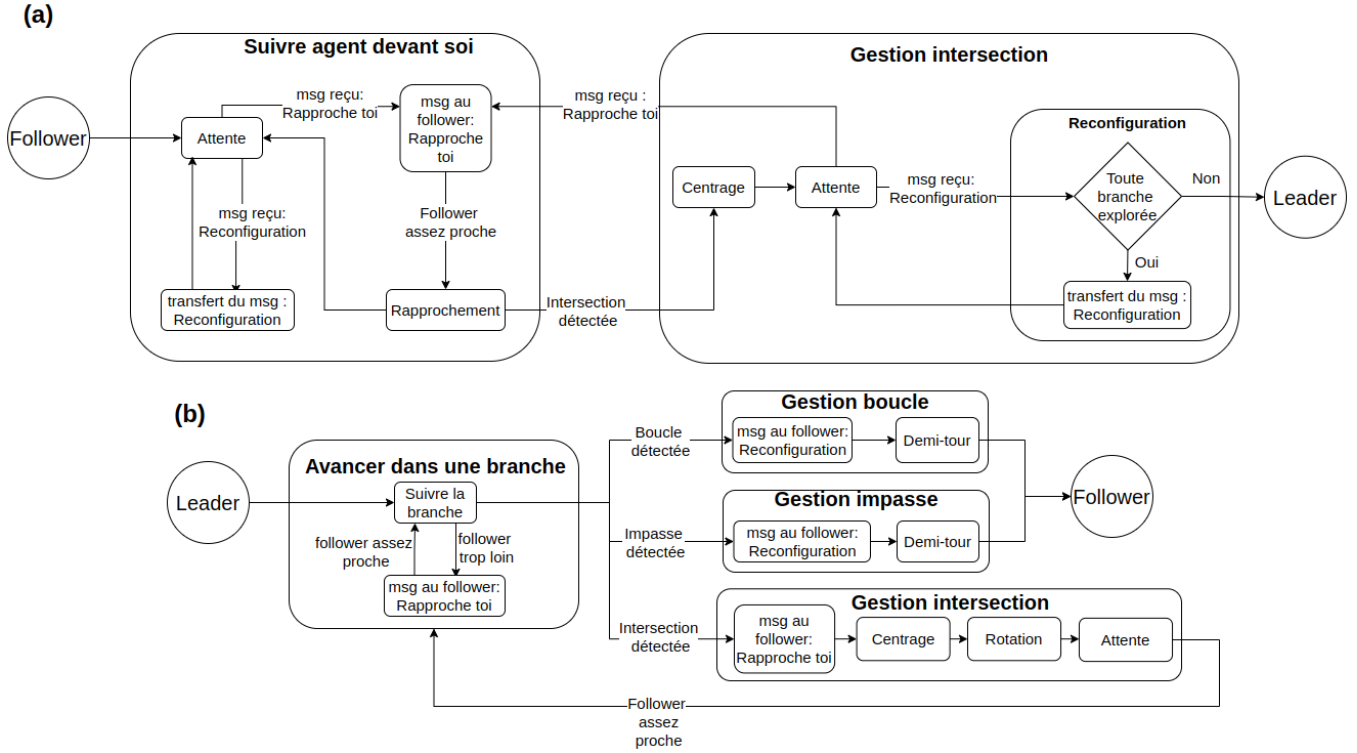


Figure 3 – (a) : machine à états du role follower (b) : machine à états du role leader

sentée dans sa version centralisée, nous allons détailler dans cette section son expression décentralisée, c'est à dire sous forme de comportements d'agents autonomes.

Dans un premier temps, nous donnons la machine à états des comportements des rôles Leader et Follower, puis nous détaillons les actions de bas niveaux ainsi que les perceptions nécessaires à leur réalisation.

5.1 Comportements relatifs aux rôles

Nous décrivons ci-après le fonctionnement des rôles Leader et Follower, qui sont formalisés dans les machines à états données en figure 3.

Le comportement du **leader** se décompose en 4 situations possibles :

1. Explorer une branche : Quand une branche a été choisie pour être explorée, suite à une intersection par exemple, ce comportement est réalisé par défaut. Il consiste à avancer tout droit jusqu'à ce qu'une nouvelle intersection ou une impasse soit atteinte. Ce comportement est réalisé en faisant toujours attention à ce que la distance entre le leader et le drone derrière lui ne soit pas trop grande.
2. Gérer une intersection : Lorsque le leader at-

teint une intersection un comportement spécifique est déclenché. Il doit envoyer un message au follower derrière lui pour qu'il se rapproche afin de pouvoir continuer l'exploration sans perte de connectivité visuelle.

3. Gérer une impasse : Quand le leader arrive dans une impasse, la chaîne de drone doit se reconfigurer par l'attribution d'un nouveau leader, suivant la stratégie présentée dans l'algorithme 2. Cela implique pour le drone leader d'envoyer un message de reconfiguration puis de devenir un follower de la chaîne.
4. Gérer une boucle : Le comportement de gestion de boucle est identique à celui d'une impasse.

Comportement d'un **follower** :

1. Suivre agent devant soi : Ce comportement permet de garantir l'hypothèse 2 de connexité de la chaîne, présentée en début de section 4.2. Son objectif est de faire en sorte qu'un follower n'avance pas tant qu'il n'en a pas reçu la demande, ni que son propre follower ne soit pas suffisamment proche de lui. Ainsi la connectivité visuelle entre les agents est maintenue en permanence.
2. Gérer les intersections : Ce comportement

permet de garantir l'hypothèse 3 de présence d'un agent en chaque intersection traversée par la chaîne. En effet les followers doivent pouvoir détecter les intersections et y rester en attente d'un message "Rapproche toi" ou "Reconfiguration de branche". Ce dernier message implique pour l'agent à l'intersection qu'il doit tester s'il devient le nouveau leader ou non.

5.2 Détails des comportements et communications

Cette section vise à mieux préciser le fonctionnement de certains comportements, en détaillant soit les perceptions nécessaires, soit les processus mis en oeuvre.

Communications entre drones : La machine à états (figure 3) utilise des messages communiqués entre drones voisins. Cependant, nous n'utilisons que deux messages différents pour gérer la stratégie ("Rapproche toi" et "Reconfiguration"). Ce nombre minimal de messages permet d'envisager son implémentation avec un système de signaux visuels très simple.

Détection des boucles : Notre machine à états permet à la chaîne de gérer les boucles de l'environnement, mais il faut être en capacité de les détecter ce qui n'est pas évident sans marquage de l'environnement ni cartographie. Nous proposons que leur détection soit basée sur la capacité des agents et notamment du leader à percevoir les autres agents. En effet, comme présenté en figure 4, si un leader perçoit devant lui un de ses followers alors il y a nécessairement une boucle dans l'environnement. Le leader entre donc dans l'état "Gestion boucle" (cf. figure 3). La présence d'un follower à l'intersection nous indique que la branche la plus à gauche a déjà été explorée et par conséquent on évite de repartir dans cette branche formant une boucle.

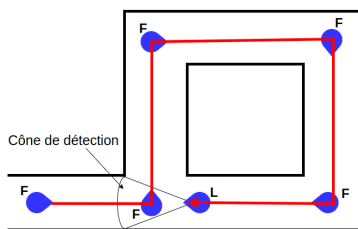


Figure 4 – Détection des boucles

Communication des demandes de reconfiguration : Les messages de reconfiguration ont

la spécificité de devoir être propagés le long de la chaîne jusqu'au futur leader. Tous les followers non positionnés aux intersections doivent transférer le même message au drone derrière eux. Mais, les followers positionnés aux intersections doivent, à la réception du message, évaluer s'ils deviennent nouveau leader. Si c'est le cas, ils ne transfèrent pas le message sinon ils le transfèrent au drone derrière eux. Ce processus est illustré en figure 5 (a) et (b).

Gestion des impasses Les impasses sont plus simples à détecter que les boucles, les capteurs informent l'agent qu'une branche se ferme. La chaîne doit donc se reconfigurer, et cela est réalisé par le même processus que celui pour les boucles (cf. ci-dessus).

Conditions pour devenir le nouveau Leader : Nous détaillons ici le processus d'auto-détermination du nouveau leader, suite à la détection d'une impasse ou d'une boucle par la chaîne. Lorsque un follower en une intersection reçoit le message "Reconfiguration" il doit déterminer s'il devient leader ou s'il continue de propager ce message. Pour ce faire, il doit déterminer si toutes les branches qu'il perçoit n'ont pas encore été explorées, car si c'est le cas il devient leader. L'agent détermine s'il reste une branche libre en réalisant une rotation sur la droite (correspondant à la prochaine branche à gauche pour l'agent qui vient de transmettre le message).

Les figures 5 (c) et (d) illustrent le cas d'un nouveau leader découvrant que la prochaine branche libre à explorer.

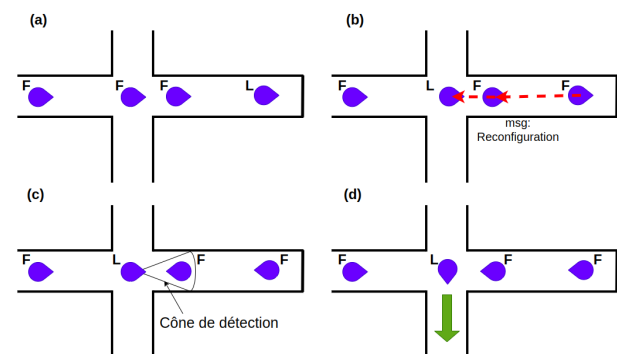


Figure 5 – Principe de détermination du nouveau leader et choix de la prochaine branche à explorer.

Sortie des agents d'une branche déjà explorée : Lorsqu'un nouveau leader a une intersection guidée la chaîne dans une nouvelle branche, les agents présents dans l'ancienne branche doivent être réintégrés à la chaîne d'ex-

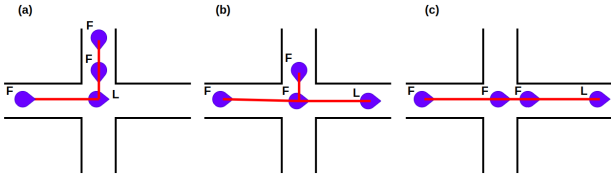


Figure 6 – Formation de chaîne en "Y"

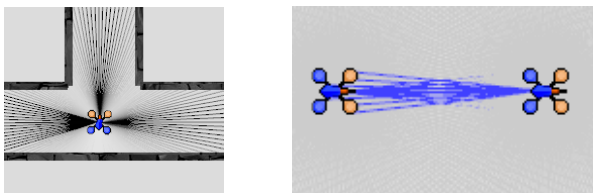
ploration. Ainsi, à l'intersection, ils seront prioritaire pour suivre le leader. Cette formation en Y au bout de la chaîne est temporaire, le temps que les agents quittent l'ancienne branche. Ce processus est illustré en figure 6.

6 Expérimentations

L'objectif est d'évaluer en simulation la stratégie d'exploration par déploiement d'une chaîne d'agents, comme définie par la machine à états en figure 3. Nous visons à mener cette évaluation dans des environnements composés de couloirs étroits, d'intersections, de boucles et d'impasses (cf section 3).

6.1 Outil de simulation

Dans cet article, on s'intéresse à l'exploration de galeries à la topologie plane, où les drones évoluent à altitude constante, représentable en 2D. Ainsi, nous avons utilisé le simulateur swarm-rescue [1] pour implémenter notre stratégie. C'est un simulateur multi-agents 2D écrit en python qui permet de simuler aisément des équipes de drones aux comportements autonomes, pour réaliser des tâches de search & rescue dans des environnements complexes créés sur mesure. La figure 8 montre un exemple d'environnement créé pour tester notre algorithme.



(a) Simu. télémètre laser (b) Localisation d'un drone voisin
Figure 7 – Capteurs

Les drones sont simulés à l'aide d'un modèle physique dynamique. Ils sont mis en mouvement par les forces qui s'appliquent sur eux, ils ont un poids, une inertie, et un corps ne les réduisant pas à des points dans l'espace.

Ils possèdent 2 capteurs, un télémètre laser permettant de récupérer la distance aux murs sur 360° et un capteur permettant de localiser les drones perçu sur 360° (cf. figure 7). Les deux capteurs fonctionnent dans un rayon limité. Ces capteurs renvoient chacun un vecteur de données bruitées. Les drones peuvent aussi s'envoyer des messages (émulant pour nous des communications par signaux, e.g., lumineux).

Le simulateur fonctionne comme suit : Au lancement, une instance de la machine à états est créée pour chaque drone puis, à chaque pas de temps, tous les drones calculent une fonction de contrôle renvoyant 3 valeurs de forces (forward, lateral, rotation) permettant leur déplacement. Les données capteurs et les échanges de messages modifient les états de la machine à états au fur et à mesure de l'exploration.

6.2 Déploiement de la chaîne de drones

Au début de la simulation, les drones sont positionnés dans un espace "de stockage" comme présenté en figure 8. A partir de cet espace ils vont être envoyés dans l'environnement pour s'ajouter à la chaîne lorsque c'est nécessaire.

La figure 9 montre le déroulement du déploiement de la chaîne dans l'environnement simulé jusqu'à la première impasse rencontrée et avant la reconfiguration de la chaîne. La première partie de la vidéo² correspond à cette simulation. Nous décrivons ci-dessous les étapes principales de cette première phase de l'exploration, c'est-à-dire jusqu'à ce que la chaîne rencontre une première impasse.

En figure 9(a) le leader a détecté une intersection, il est donc dans l'état "Gestion Intersection" (cf. figure 3). La pointe bleue indique la direction vers laquelle le leader se dirige, c'est à

2. <https://mathismbl.github.io/swarm-rescue/>

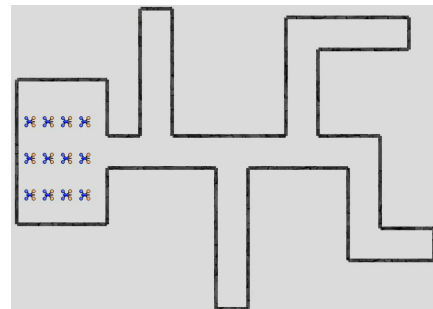


Figure 8 – Début de la simulation.

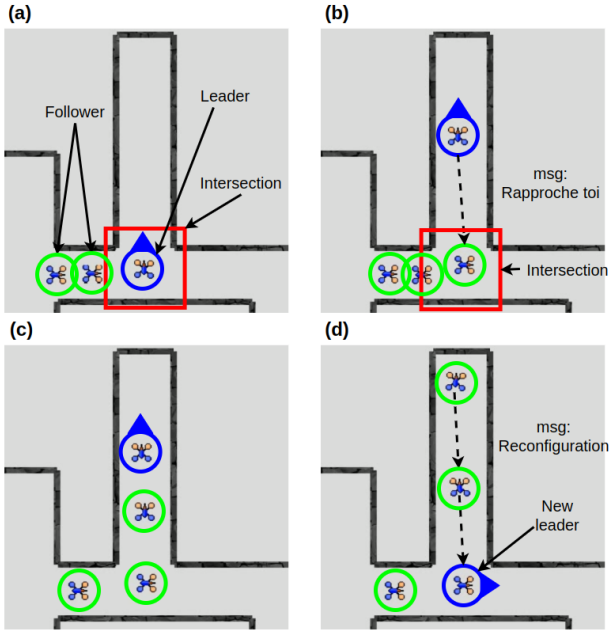


Figure 9 – Déploiement de la chaîne jusqu'à la première impasse rencontrée

dire la branche la plus à gauche, en accord avec notre stratégie d'exploration. Cette intersection étant au début de l'environnement, deux followers sont suffisants dans la chaîne pour maintenir une connexion avec l'entrée.

En (b) le drone leader est désormais dans l'état "Avancer dans une branche" et le drone qui le suit est passé en "Gestion Intersection". Mais le leader est trop loin du reste de la chaîne. Pour continuer son exploration, il envoie un message "Rapproche toi", perçu par le follower positionné à l'intersection qui va lui aussi envoyer ce message au drone le suivant. Ceci permet de faire en sorte que chaque agent ne se déplace que si un autre agent se trouve à proximité derrière lui, garantissant la connectivité.

On observe en (c) que le drone leader perçoit un follower suffisamment proche de lui pour poursuivre l'exploration de la branche et donc repasser dans l'état "Suivre la branche" (cf. figure 3). La situation en (d) présente la première rencontre par la chaîne d'une impasse. Le drone ayant détecté l'impasse, a propagé un message "Reconfiguration" et quitté son rôle de leader. Le drone positionné à l'intersection a pu s'auto-déterminer nouveau leader car l'intersection possède des branches inexplorées.

6.3 Reconfiguration de la chaîne

Dans cette dernière section nous présentons la suite du processus d'exploration, qui consiste en

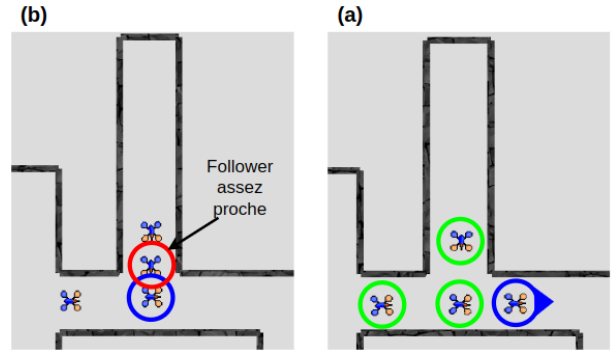


Figure 10 – Evolution de la chaîne à l'intersection du nouveau leader (en bleu) : a) les agents dans la branche déjà explorée se sont rapprochés du nouveau leader, b) le nouveau leader progresse, le bout de la chaîne prend la forme d'un 'Y' le temps que ces agents le suivent.

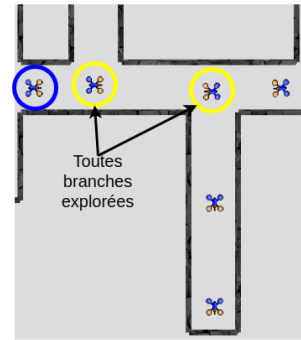


Figure 11 – Fin de l'exploration, la dernière branche explorée implique que l'agent à l'entrée devient le leader.

la reconfiguration de la chaîne suite à la détection d'une impasse.

Le nouveau leader qui s'est auto-désigné à l'intersection va maintenant guider la chaîne. La figure 10 (a) montre la situation où tous les agents de la chaîne se sont rapprochés du nouveau leader pour qu'il puisse poursuivre l'exploration. Dans la figure 10 (b) la chaîne est dans une phase transitoire où son extrémité forme un "Y" le temps que les agents présents dans l'ancienne branche explorée se réinsèrent correctement dans la chaîne (le mécanisme de gestion de cette phase transitoire n'est pas détaillé pour des raisons de place dans cet article).

La figure 11 montre la fin de l'exploration. Lorsque la dernière branche de l'environnement a été explorée, le message de reconfiguration, envoyé par le leader ayant rencontré la dernière impasse, remonte jusqu'à l'agent situé à l'entrée de l'environnement. Cet agent devient donc le nouveau leader et l'exploration est terminée en accord avec la condition de terminaison définie dans l'algorithme 2.

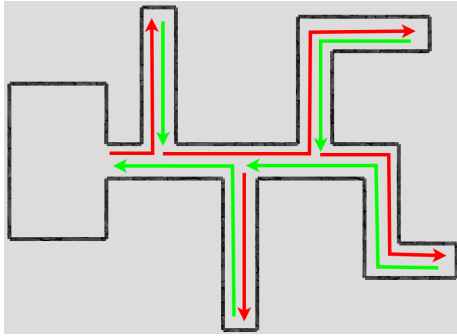


Figure 12 – Chemin suivi par le leader : en rouge les phases d’avancement dans les couloirs, en vert les messages de demande de reconfiguration pour l’allocation d’un nouveau leader.

Ainsi le chemin d’exploration (suivi par les agents leader) est représenté en figure 12. Les flèches rouge représentent le trajet physiquement parcouru par le drone possédant le rôle de leader. Et les flèches vertes représentent le trajet des messages de reconfiguration permettant de transférer le rôle de leader à un autre agent.

7 Conclusion

Dans cet article, nous avons proposé un algorithme garantissant l’exploration complète de tout environnement souterrain composé de tunnels reliés arbitrairement. L’exploration est effectuée par une flotte de mini-drones formant une chaîne dynamique, composée de drones suiveurs situés entre un drone leader (guidant l’exploration) et la base. Cette approche permet la transmission d’informations du leader vers la base en utilisant uniquement des signaux visuels (aucun réseau de communication sans fil n’est requis) et fournit une représentation topologique de l’environnement sans le cartographier. La stratégie d’exploration repose sur l’extension de la recherche "par la main sur le mur". La chaîne d’agents est ainsi capable de détecter les impasses et les boucles, et de se reconfigurer en déterminant un nouveau leader dynamiquement. Nous avons présenté les premiers résultats expérimentaux à l’aide d’un simulateur réaliste de mini-drones quadricoptère.

Dans les travaux futurs, nous visons à améliorer le comportement leader-follower afin d’assurer une progression plus fluide de la chaîne et à étendre l’approche proposée à des scénarios d’exploration multi-leader (formant un arbre d’exploration dynamique depuis l’entrée). De plus, il est envisagé d’autoriser les drones à se séparer, momentanément et dans des situations spécifiques, du reste de la chaîne, en vue de li-

miter le nombre de drones nécessaires à l’exploration. Nous avons également commencé à construire un démonstrateur avec drones réels, en vue de préparer des missions réelles dans le cadre du projet ANR VORTEX³.

Références

- [1] E. Battesti. Swarm rescue simulator. <https://github.com/emmanuel-battesti/swarm-rescue>.
- [2] E. Bienias, K. Szczepański, and P. Duch. Maze Exploration Algorithm for Small Mobile Platforms. *Image Processing & Communications*, 21(3) :15–26, Sept. 2016.
- [3] T. Dang, M. Tranzatto, S. Khattak, F. Mascarich, K. Alexis, and M. Hutter. Graph-based subterranean exploration path planning using aerial and legged robots. *Journal of Field Robotics*, 37(8) :1363–1388, Dec. 2020.
- [4] M. Dharmadhikari, H. Nguyen, F. Mascarich, N. Khedekar, and K. Alexis. Autonomous Cave Exploration using Aerial Robots. In *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 942–949, June 2021. ISSN : 2575-7296.
- [5] N. Evangeliou, D. Chaikalis, A. Tsoukalas, and A. Tzes. Visual collaboration leader-follower uav-formation for indoor exploration. *Frontiers in Robotics and AI*, 8 :777535, 2022.
- [6] A. Farooq, A. Anastasiou, N. Souli, C. Laoudias, P. S. Kolios, and T. Theodoridis. UAV Autonomous Indoor Exploration and Mapping for SAR Missions : Reflections from the ICUAS 2022 Competition. In *2022 19th International Conference on Ubiquitous Robots (UR)*, pages 621–626, July 2022.
- [7] A. Franchi, L. Freda, G. Oriolo, and M. Vendittelli. The sensor-based random graph method for cooperative robot exploration. *IEEE/ASME Transactions on Mechatronics*, 14(2) :163–175, 2009.
- [8] N. Gauville and F. Charpillet. Approche locale pour l’exploration autonome d’environnements inconnus par une flottille de robots. *Revue Ouverte d’Intelligence Artificielle*, 3(5-6) :625–643, 2022.
- [9] S. A. Ghauri, M. Sarfraz, R. A. Qamar, M. F. Sohail, and S. A. Khan. A review of

3. ANR VORTEX 2023-28, référence ANR-23-IAS2-0005

- multi-uav task allocation algorithms for a search and rescue scenario. *Journal of Sensor and Actuator Networks*, 13(5), 2024.
- [10] C. Gong, S. Tully, G. Kantor, and H. Cho-set. Multi-agent deterministic graph mapping via robot rendezvous. In *2012 IEEE International Conference on Robotics and Automation*, pages 1278–1283, 2012.
- [11] P. Laclau, V. Tempez, F. Ruffier, E. Natalizio, and J.-B. Mouret. Signal-Based Self-Organization of a Chain of UAVs for Subterranean Exploration. *Frontiers in Robotics and AI*, 8 :614206, Apr. 2021.
- [12] R. Lösch, S. Grehl, M. Donner, C. Buhl, and B. Jung. Design of an Autonomous Robot for Mapping, Navigation, and Manipulation in Underground Mines. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1407–1412, Oct. 2018. ISSN : 2153-0866.
- [13] K. Otsu, S. Tepsuporn, R. Thakker, T. S. Vaquero, J. A. Edlund, W. Walsh, G. Miles, T. Heywood, M. T. Wolf, and A.-A. Agha-Mohammadi. Supervised Autonomy for Communication-degraded Subterranean Exploration by a Robot Team. In *2020 IEEE Aerospace Conference*, pages 1–9, Mar. 2020. ISSN : 1095-323X.
- [14] P. Petráček, V. Krátký, M. Petrlík, T. Báča, R. Kratochvíl, and M. Saska. Large-Scale Exploration of Cave Environments by Unmanned Aerial Vehicles. *IEEE Robotics and Automation Letters*, 6(4) :7596–7603, Oct. 2021. Conference Name : IEEE Robotics and Automation Letters.
- [15] R. Rafifandi, D. Asri, and E. Ekawati. Leader–follower formation control of two quadrotor uavs. *SN Applied Sciences*, 1 :539, 2019.
- [16] D. Silver, D. Ferguson, A. Morris, and S. Thayer. Topological exploration of subterranean environments. *Journal of Field Robotics*, 23(6-7) :395–415, 2006.